Join our community, grow your knowledge and learn from others!
Sign In
Don't have an Epic Games account? Sign up

Network Emulation

Emulate network packet lag and loss in Unreal Engine.

ON THIS PAGE
Enable Network Emulation
Unreal Editor
Command-Line
Console
Configuration Files
Settings Reference
Recommendations for Use

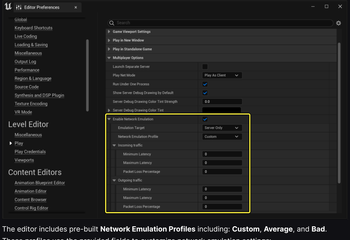
Testing multiplayer on a single machine requires you to run multiple instances of your game, with one acting as a server and the others acting as clients. Normally, this creates an environment with ideal conditions for your replicated data. Your replicated data will be passed through the appropriate systems, similar to a real networked environment, but there will be no lag or packet loss, which means you won't get an accurate idea of how it will behave when deployed to end-users. Similarly, LAN-based tests will not experience these conditions.

Network Emulation simulates lag and packet loss for servers and clients. This is crucial in identifying issues that may arise in networking environments. The Network Emulation settings are configurable in the **Unreal Editor**, command-line, console, and configuration files.

Enable Network Emulation

Unreal Editor

To use Network Emulation, from the **Menu Bar** navigate to **Edit > Editor Preferences > Level Editor > Play > Multiplayer Options** and enable the **Enable Network Emulation** checkbox. This enables the Network Emulation settings.



The editor includes pre-built **Network Emulation Profiles** including: **Custom**, **Average**, and **Bad**. These profiles use the provided fields to customize network emulation settings:

- **Incoming Traffic**: cause a delay or loss for receiving a packet
  - **Minimum Latency**: minimum amount of time lag in milliseconds (corrsponds to `PktIncomingLagMin`)
  - **Maximum Latency**: maximum amount of time lag in milliseconds (corresponds to `PktIncomingLagMax`)
  - **Packet Loss Percentage**: chance that a packet is lost before being received (corresponds to `PktIncomingLoss`)
- **Outgoing Traffic**: cause a delay or loss for sending a packet
  - **Minimum Latency**: minimum amount of time lag in milliseconds (corresponds to `PktLagMin`)
  - **Maximum Latency**: maximum amount of time lag in milliseconds (corresponds to `PktLagMax`)
  - **Packet Loss Percentage**: chance that a packet is lost before being sent (corresponds to `PktLoss`)

Command-Line

You can specify settings with a command-line argument using the syntax:

```
<SETTING_NAME>=<VALUE>
```

Copy full snippet

Console

You can specify settings from the console at runtime using the syntax:

```
NetEmulation.<SETTING_NAME> <VALUE>
```

Copy full snippet

Configuration Files

To set these values from your engine configuration files such as `DefaultEngine.ini`, add the `[PacketSimulationSettings]` section to your your configuration file, followed by the `<SETTING_NAME>=<VALUE>` pairs you wish to specify:

```
[PacketSimulationSettings]
<SETTING_NAME>=<VALUE>
<SETTING_NAME>=<VALUE>
...
```

Copy full snippet

Settings Reference

These settings provide you with a multitude of combinations to customize your network emulation:

Setting	Description	Value
`Off`	Turn off network emulation	Toggle command, no value required.
`PktLoss`	Specifies a percentage chance of an outbound packet being discarded to simulate packet loss.	Clamped value between 0 (no packets dropped) and 100 (all packets dropped).
`PktOrder`	Changes the order of packets at random. This works by randomly selecting packets to be delayed until a subsequent call to `FlushNet`.	0 = False, Anything else = True
`PktDup`	Specifies a percentage chance of a sent outbound packet being duplicated.	Clamped value between 0 (no packets duplicated) and 100 (all packets duplicated).
`PktLag`	Delays the sending of a packet by the amount of time specified in milliseconds. **Note**: Cannot be used with `PktOrder`.	Value in milliseconds.
`PktLagVariance`	Causes packet lag to be a random variable instead of constant time in milliseconds. If set, lag becomes a random variable from `[PktLag - PktLagVariance, PktLag + PktLagVariance]`. **Note**: Can only be enabled when `PktLag` is enabled.	Value is chosen at random from the range `[-Variance, Variance]`.
`PktLagMin`	Causes packets to have a minimum amount of lag.	Value in milliseconds.
`PktLagMax`	Causes packets to have a maximum amount of lag.	Value in milliseconds.
`PktIncomingLagMin`	Causes packets to have a minimum amount of lag for the receiving end.	Value in milliseconds.
`PktIncomingLagMax`	Causes packets to have a maximum amount of lag for the receiving end.	Value in milliseconds.
`PktIncomingLoss`	Specifies a percentage chance of an inbound packet being discarded to simulate packet loss.	Clamped value between 0 (no packets dropped) and 100 (all packets dropped).
`PktJitter`	Causes sent packets to have a variable fluctuating latency by adding the value of `PktJitter` to range of possible packet lag times.	Value in milliseconds.
`PktEmulationProfile`	Set a predefined emulation profile.	Name of profile.

Recommendations for Use

Multiplayer games running with perfect connectivity will not experience many bugs that only occur during poor connectivity. To ensure that you catch these bugs, you should perform local and LAN-based tests with extremely harsh conditions. For example:

- 500 round trip ping.
- 10% packet loss or higher.

Epic Games uses conditions like these to test cooperative and competitive gameplay. This makes it possible to catch numerous bugs and exploits that would otherwise go overlooked, and strongly incentivizes optimizing network performance.